PERSÖNLICHES ENTWICKLUNGSPROJEKT

Eine Idee. Viele Möglichkeiten. Dein eigener Code.

Entwicklungsplan

für den persönlichen Alltagsplaner

Von der ersten HTML-Seite bis zur vollständig gehosteten Webanwendung – mit Lernen, Alltagsorganisation, Finanzen und Widgets.

7 Entwicklungsabschnitte

25 Schritte

Vom Tab S8 zum Heimserver

KONZEPT & ENTWICKLUNGSPLAN · OKTOBER 2026

DEIN WEG DURCH DAS PROJEKT

Inhaltsübersicht

Sieben aufeinander aufbauende Phasen – von den ersten Programmiergrundlagen bis zu deinen eigenen Widgets.

Überblick & Entwicklungsreihenfolge 3

Abschnitt 01 · Das Fundament: Programmieren lernen 4

Abschnitt 02 · Aus der Übungsseite wird eine richtige Anwendung 6

Abschnitt 03 · Der vollständige Alltagsplaner 8

Abschnitt 04 · Das integrierte Lernsystem 10

Abschnitt 05 · Ausbildung, Berufsschule, Job und Praktikum 12

Abschnitt 06 · Die Anwendung wird intelligent und zusammenhängend 13

Abschnitt 07 · Jetzt lohnen sich die Widgets 14

Technologien und Lernreihenfolge 16

Umsetzungsprinzipien 16

Der nächste Schritt 17

Der Plan ist eine Reihenfolge von Abhängigkeiten und Lernzielen, kein fester Zeitplan.

Das Projekt gliedert sich in sieben aufeinander aufbauende Entwicklungsabschnitte. Zuerst werden die Programmiergrundlagen gelernt; umfangreichere Funktionen folgen, sobald die technische Basis vorhanden ist.

DAS PROJEKTZIEL

Eine vollständige, selbst entwickelte Webanwendung

Mit Alltag, Kalender, Bewerbungen, Finanzen, Lernen, Ausbildung, Arbeit und später den passenden Widgets für Flip8 und Watch8.

Entwicklung mit Tablet oder PC · Hosting und private Datenspeicherung auf einem geschützten Heimserver · Eigenständige Programmierung

Die beiden Projekte bleiben getrennt: **Der Alltagsplaner ist nicht Evoluna.** Er übernimmt einige praktische Lernideen daraus, aber keine Geschichte, Rivalen oder Rollenspielmechaniken. Ducky AI wiederum bleibt dein eigenes langfristiges KI-Projekt, das später angebunden werden kann.

Die gesamte Entwicklungsreihenfolge

Abschnitt	Name	Inhalt	Schritte
01	**Das Fundament**	Entwicklungsumgebung, HTML, CSS und JavaScript	1–5
02	**Eine echte Webanwendung**	Struktur, Server, Datenbank und Dokumentenarchiv	6–10
03	**Dein Alltagssystem**	Kalender, Fahrtwege, Finanzen und Bewerbungen	11–14
04	**Dein Lernsystem**	Lernmethoden, Prüfungen, S Pen, Programmieren und Sprachen	15–18
05	**Dein Berufsleben**	Ausbildung, Berufsschule, Job und Praktikum	19–20
06	**Alles arbeitet zusammen**	Dashboard, Verknüpfungen, Automatisierung und KI-Schnittstellen	21–22
07	**Widgets und Fertigstellung**	Zuerst Flip8, dann Watch8, danach Abschlussprüfung	23–25

Das ist eine **Abhängigkeitsreihenfolge, kein starrer Kalenderplan**. Manche Schritte werden wenige Übungseinheiten brauchen; andere – insbesondere die Lernplattform, die Sprachauswertung und die KI-Anbindung – sind eigenständige größere Entwicklungsaufgaben.

Im folgenden Plan steht bei jedem Schritt, was du lernst, was entwickelt wird und wann du weitergehen kannst.

ABSCHNITT 01 · ENTWICKLUNGSPHASE

Das Fundament: Programmieren lernen

Ziel: Du kannst selbst einfache Webseiten erstellen, gestalten und interaktiv machen.

01 Die Entwicklungswerkstatt einrichten

Erstelle dein privates GitHub-Repository `alltagsplaner-prototyp` und richte einen Browser-Codeeditor ein, beispielsweise GitHub Codespaces. Du arbeitest auf deinem Galaxy Tab S8 mit Tastatur.

Hier lernst du, was ein Projektverzeichnis ist, wie man Dateien erstellt, Änderungen speichert und mit GitHub nachvollziehbar versioniert. Lege auch eine `README.md` an, in der die Projektziele festgehalten werden.

Abgeschlossen, wenn: Du deine eigenen Dateien bearbeiten, speichern und eine Änderung auf GitHub wiederfinden kannst.

02 HTML: Das Grundgerüst

Beginne mit `index.html`. Du lernst Überschriften, Absätze, Eingabefelder, Schaltflächen, Formulare und die grundlegende Seitenstruktur kennen.

Die erste Oberfläche enthält einen Titel und einen kleinen Aufgabenbereich. Noch muss sie nicht besonders schön sein.

Abgeschlossen, wenn: Du ohne Vorlage eine einfache HTML-Seite mit einem Eingabefeld und einer Schaltfläche aufbauen kannst.

03 CSS: Ein eigenes Designsystem

Jetzt gestaltest du die Webseite in Perlweiß, Hellrosa, Altrosa, Silber und Pflaume. Du lernst Abstände, Schriftgrößen, Farben, Karten, Layouts und responsive Darstellung.

Wichtig ist ein **einheitliches Designsystem**: Eine Schaltfläche soll im Kalender später genauso aussehen und funktionieren wie im Finanzbereich. Dazu gehört auch die Unterstützung von Touchscreen, Tastatur und S Pen.

Abgeschlossen, wenn: Deine erste Seite auf Tablet und Smartphone übersichtlich aussieht und du Änderungen am Design selbst vornehmen kannst.

04 JavaScript: Die Webseite reagiert

Du lernst Variablen, Funktionen, Bedingungen, Arrays, Objekte und Ereignisse. Danach verbinden wir unsere HTML-Elemente mit JavaScript.

Du verstehst beispielsweise, wie ein Klick auf „Aufgabe hinzufügen“ dazu führt, dass eine neue Aufgabe auf dem Bildschirm erscheint.

Abgeschlossen, wenn: Du selbst eine interaktive Funktion programmieren und einfache Fehler mithilfe der Entwicklerwerkzeuge finden kannst.

05 Das erste fertige Mini-Projekt: Aufgabenverwaltung

Hier verbinden wir alles bisher Gelernte zu einem ersten tatsächlich nutzbaren System.

Die Aufgabenverwaltung bekommt Hinzufügen, Bearbeiten, Löschen, Abhaken, Fälligkeitsdaten, eigene Kategorien, Filter und Sortierung. **Priorität und Dringlichkeit bleiben zwei getrennte Eigenschaften**: Wichtigkeit sagt etwas anderes aus als zeitlicher Handlungsdruck.

Außerdem sollen wiederkehrende Aufgaben, Notizen und verschiedene Ansichten später erweiterbar sein.

Abgeschlossen, wenn: Du Aufgaben vollständig verwalten kannst und wir unsere erste kleine Anwendung tatsächlich benutzen können.

Meilenstein 1 – Deine erste funktionierende Webanwendung

Du hast selbst HTML, CSS und JavaScript eingesetzt und eine praktische Aufgabenverwaltung entwickelt.

ABSCHNITT 02 · ENTWICKLUNGSPHASE

Aus der Übungsseite wird eine richtige Anwendung

Ziel: Deine Anwendung wird dauerhaft nutzbar, erweiterbar und auf eurem Heimserver betreibbar.

06 Daten speichern und wieder laden

Zunächst lernen wir mit unkomplizierten Testdaten, wie eine Anwendung ihren Zustand speichert, beispielsweise mit `localStorage`. Dabei üben wir auch den Export und Import von Daten.

Das ist nur eine Zwischenstufe. Persönliche Verträge, Bewerbungsdokumente und Finanzdaten wollen wir nicht dauerhaft als ungeschützte Browserdaten verwalten.

Abgeschlossen, wenn: Deine Testaufgaben nach dem Schließen und erneuten Öffnen des Browsers noch vorhanden sind.

07 Eine übersichtliche Projektarchitektur entwickeln

Jetzt lernen wir, den Code in gut verständliche Dateien und Funktionen aufzuteilen. Wir erstellen die Navigation und gemeinsame Komponenten für Eingabefelder, Kategorien, Dialoge und Übersichten.

Unsere Webanwendung soll wachsen können, ohne dass wir später alles neu schreiben müssen. Hier können wir auch entscheiden, wann TypeScript oder ein Frontend-Framework wie React tatsächlich hilfreich wird – **nicht schon am allerersten Lerntag**.

Abgeschlossen, wenn: Neue Funktionen ergänzt werden können, ohne die bestehende Aufgabenverwaltung durcheinanderzubringen.

08 Die erste Version auf dem Heimserver veröffentlichen

Sobald die Hosting-Umgebung vorbereitet ist, veröffentlichen wir eine lauffähige Version auf einem geeigneten Linux-Server.

Eine eigene Domain und ein abgesicherter Zugriffsweg werden eingerichtet. Hosting, Domainverwaltung und Datensicherung erfolgen mit klaren Zuständigkeiten.

Abgeschlossen, wenn: Unsere Testwebseite über eure vorgesehene Adresse aufgerufen werden kann.

09 Backend, Datenbank und Benutzeranmeldung

Bisher läuft die Logik hauptsächlich im Browser. Jetzt ergänzen wir einen Serverteil, der Anfragen verarbeitet und Daten dauerhaft speichert.

Wir lernen die Grundlagen von APIs, Datenbanken, Benutzeranmeldungen und Berechtigungen. Als mögliche Technik kommt etwa ein JavaScript-/TypeScript-Backend mit einer relationalen Datenbank infrage.

Wir verwenden klare Datenmodelle für Aufgaben, Personen, Termine, Verträge und Verknüpfungen. Später sollen weitere Benutzer getrennte private Daten und gegebenenfalls gemeinsame Bereiche haben können.

Abgeschlossen, wenn: Du dich anmelden kannst und dieselben Aufgaben auf Tablet und Smartphone angezeigt bekommst.

10 Das private Dokumentenarchiv

Jetzt setzen wir deinen ausdrücklichen Speicherwunsch um.

Alle dauerhaften Originaldokumente bleiben auf dem Heimserver. Die Webseite ist lediglich die Oberfläche für berechtigten Zugriff. Dort kannst du Dateien hochladen, betrachten, organisieren und herunterladen.

Dazu gehören insbesondere Verträge, Zeugnisse, Bewerbungsunterlagen, Schulmaterialien, Zertifikate, Lohnabrechnungen und Lernnachweise. Die Dateien werden nicht in einem öffentlich erreichbaren Webseitenverzeichnis abgelegt. Beim Anzeigen können temporäre Daten auf das verwendete Gerät übertragen werden.

Wir entwickeln Kategorien, Dateizuordnungen und Zugriffskontrollen. Die Infrastruktur erhält eine getrennte, regelmäßig überprüfte Datensicherung.

Abgeschlossen, wenn: Du ein privates Dokument hochladen, einem passenden Eintrag zuordnen und später auf einem anderen Gerät wieder abrufen kannst.

Meilenstein 2 – Der Planer läuft auf eurem Server

Ab hier haben wir eine eigene Webanwendung mit dauerhafter Datenspeicherung, Benutzerzugriff und einem privaten Dokumentenarchiv.

ABSCHNITT 03 · ENTWICKLUNGSPHASE

Der vollständige Alltagsplaner

Ziel: Kalender, Aufgaben, Bewerbungen und Finanzen bilden bereits ein zusammenhängendes Alltagssystem.

11 Der umfangreiche Kalender

Wir entwickeln einen vollwertigen Kalender, nicht bloß eine chronologische Aufgabenliste.

Er umfasst Tages-, Wochen- und Monatsansichten, eigene beliebig benennbare und farblich gestaltbare Kategorien, einzeln ein- und ausblendbare Kalender, einmalige und wiederkehrende Termine sowie Ausnahmen von Wiederholungsregeln.

Dazu kommen Terminnotizen, Uhrzeiten, Orte, Erinnerungen, unterschiedliche Terminarten und Verknüpfungen zu Aufgaben. Auch Reisen, Konzerte, Trainingseinheiten und private Termine sollen problemlos zusammenpassen.

Besonders wichtig: Ein geplanter Termin, eine bestätigte Buchung und eine bereits stattgefundene Veranstaltung dürfen nicht durcheinandergeraten.

Abgeschlossen, wenn: Du deinen tatsächlichen Wochen- und Monatsablauf verlässlich im Kalender organisieren kannst.

12 Wege, Reiseplanung und intelligente Terminregeln

Jetzt erweitern wir den Kalender um reale Alltagslogik.

Ein Termin kann eine Adresse, Anfahrtszeit, Abfahrtszeit von zu Hause und eine geplante Rückkehr enthalten. Wir berücksichtigen dein Zeitpolster für den Weg zum Bahnhof und können Fahrten zwischen Wohnung, Sport, Familie und anderen Zielen abbilden.

Zusätzlich planen wir:

- Fahrten mit ÖPNV, Auto oder zu Fuß, inklusive Verlinkung zur Navigation.
- Reiseabschnitte mit mehreren Stationen sowie Buchungs- und Zahlungsstatus.
- Erkennen von Zeitüberschneidungen und zu knappen Übergängen.
- Automatische Aussetzung wiederkehrender Trainingstermine während eingetragener Reisen – ohne die eigentliche Terminserie zu löschen.
- Optionale Aktivitäten wie Spaziergänge, die sich an den vorhandenen Terminen orientieren.

Live-Verbindungen und Verspätungen benötigen später eine geeignete externe Fahrplanauskunft; die Anwendung soll auch ohne diese Schnittstelle mit manuell gepflegten Reisezeiten funktionieren.

Abgeschlossen, wenn: Dein Kalender nicht nur zeigt, *wann* ein Termin stattfindet, sondern auch, wann du voraussichtlich losmusst und wieder zurück bist.

13 Die vollständige Finanzverwaltung

Hier legen wir ein allgemeines Finanzmodell mit nachvollziehbaren Buchungen und bestätigten Daten an.

Der Finanzbereich erhält Einnahmen und Ausgaben, Konten, Fixkosten, Abonnements, Verträge, Ratenzahlungen, Rückzahlungen, Sparziele, Reisebudgets, Monatsprognosen und Auswertungen. **Euro und Schweizer Franken** werden getrennt geführt und nur bei Bedarf mit ausgewiesenem Wechselkurs umgerechnet.

Jeder Posten bekommt einen eindeutigen Status, zum Beispiel vorgemerkt, bestätigt, gebucht, bezahlt oder storniert. Wir unterscheiden auch tatsächlich angespartes Geld von bloß geplanten Sparraten. Vertragsänderungen erhalten ihre jeweiligen Start- und Endtermine.

Zur Ersteinrichtung werden ausschließlich neutrale Beispieldaten verwendet. Das Datenmodell verarbeitet laufende Ausgaben, Einmalzahlungen, Abonnements, Sparziele und veränderliche Vertragskosten.

Unbestätigte Anschaffungen, mögliche neue Verträge und zukünftige Einnahmen erscheinen ausschließlich als getrennte Szenarien. Historische Annahmen werden nicht als Ist-Daten übernommen.

Später werden Lohnabrechnungen aus dem Arbeitsbereich, Ausbildungsvergütungen und mit Reisen verknüpfte Ausgaben automatisch vorgeschlagen beziehungsweise zugeordnet. **Erwartete Einnahmen sind dabei nicht gleich eingegangene Zahlungen.**

Abgeschlossen, wenn: Du eine vollständige, centgenaue Finanzübersicht erhältst und die Auswirkungen von Vertragsänderungen auf zukünftige Monate erkennen kannst.

14 Der Bewerbungsmanager

Jetzt erweitern wir unsere Aufgabenverwaltung um Bewerbungsprozesse für Ausbildungsplätze, Praktika und Jobs.

Jede Bewerbung erhält einen Datensatz mit Unternehmen, Stelle, Ausbildungs- oder Beschäftigungsart, Arbeitsort, Ausschreibungslink, Bewerbungsfrist, Priorität, Dringlichkeit, Dokumenten und Notizen.

Wir integrieren Statuswechsel von der Merkliste über Vorbereitung und Versand bis hin zu Rückmeldungen, Auswahlverfahren, Zusage oder Absage. Hinzu kommen eine Kanban-Ansicht, Wiedervorlagen, Gesprächsvorbereitung und Verknüpfungen zum Kalender.

Bestehende Bewerbungen können anhand ihres bestätigten Versanddatums, ihrer Rückmeldungen und Fristen als Datensätze übernommen werden.

Abgeschlossen, wenn: Du alle Bewerbungen verwalten kannst und sofort erkennst, welche nächste Handlung notwendig ist – ohne bereits abgeschickte Bewerbungen fälschlich als noch offen anzuzeigen.

Meilenstein 3 – Der Planer wird im Alltag brauchbar

Zu diesem Zeitpunkt haben wir bereits genügend echte Inhalte für ein schönes Dashboard und erste sinnvolle Widgets. Wir verschieben die native Widget-Entwicklung aber noch etwas, bis auch das Lernsystem angebunden ist.

ABSCHNITT 04 · ENTWICKLUNGSPHASE

Das integrierte Lernsystem

Ziel: Ein einziger Lernbereich, der unterschiedliche Lernmethoden, Prüfungen, Handschrift, Programmierung und Sprachen verbindet.

15 Die persönliche Lernbibliothek

Wir erstellen die Grundlage des Lernbereichs: frei anlegbare Fächer, Themen, Unterthemen, Lernziele, Materialsammlungen, Notizen, Lernpläne und individuelle Lernpfade.

Alle Lerninhalte gehören zu einem gemeinsamen System. Es gibt **keine getrennten Hauptmenüs für Quizlet, Turm und Runenschmiede**.

Jedes Thema kann einen Lernzustand durchlaufen:

Neu lernen → Wiederholen → Festigen → Beweisen.

Bereits vorhandenes Wissen kann durch Einstufungsaufgaben berücksichtigt werden. Lernfortschritt soll auf tatsächlich gezeigten Fähigkeiten beruhen, nicht auf künstlichen Erfahrungspunkten.

Abgeschlossen, wenn: Du selbst ein neues Fach anlegen und dessen Themen strukturiert lernen und verfolgen kannst.

16 Lernmethoden, Prüfungen und der S Pen

In diesem Schritt entwickeln wir den großen Funktionsumfang unseres Lernsystems.

Wir ergänzen Karteikarten, verschiedene Abfrageformen, Multiple Choice, schriftliche Antworten, Zuordnungsspiele, Lernspiele, Wiederholungsalgorithmen, Bilder, Audio, Lernstatistiken und Import-/Exportfunktionen – mit den umfangreichen Quizlet-ähnlichen Möglichkeiten als Ziel.

Dazu kommen die praktischen Prüfungsfunktionen aus unserem Turmkonzept: Einstufungen, Wissensprüfungen, anpassbare Themenebenen, pausierbare Prüfungen, Fehleranalysen, Auffrischungen und langfristig erhaltene Wissensnachweise. **Ohne Geschichte, Rivalen oder NPC-Kämpfe.**

Ein großer Entwicklungsschritt ist der S Pen. Wir bauen echte handschriftliche Eingabeflächen mit Schreiben, Radieren, Rückgängig und Speicherung. Später ergänzen wir das Beschriften von Bildern und Arbeitsblättern, PDF-Anmerkungen sowie eine mögliche Handschrifterkennung.

Alles wird demselben Lernthema zugeordnet: Ob du eine Karteikarte beantwortest, etwas handschriftlich erklärst oder eine Prüfung ablegst, der Lernfortschritt bleibt zusammenhängend.

Abgeschlossen, wenn: Du auf deinem Tab S8 Lerninhalte durchlesen, mit Karteikarten üben, handschriftlich bearbeiten, prüfen und als Nachweis speichern kannst.

17 Programmieren und technische Übungen integrieren

Jetzt ergänzen wir die Funktionen der Runenschmiede, allerdings **innerhalb technischer Lernthemen** und nicht als separates Hauptsystem.

Für Fächer wie JavaScript, Python oder Logik entwickeln wir einen Code-Editor, praktische Programmieraufgaben, Fehleranalysen, Projektübungen und Tests.

Später können wir sichere Ausführungsumgebungen verwenden, um Lösungen zu kontrollieren. Dabei muss der ausgeführte Code von sensiblen Serverdaten isoliert bleiben.

Das Ziel ist, dass du nicht nur theoretische Fragen zur Programmierung beantwortest, sondern dein Wissen auch wirklich anwendest. Ein bestandenes Praxisprojekt kann gleichzeitig als Lernnachweis dienen.

Abgeschlossen, wenn: Du beispielsweise JavaScript innerhalb eines Lernthemas lernen, eine passende Aufgabe programmieren und deine Lösung überprüfen kannst.

18 Vier Sprachen vollständig integrieren

Englisch, Spanisch, Koreanisch und Japanisch werden als normale Fächer in unserem Lernsystem angelegt.

Alle vier Sprachen nutzen dieselben grundlegenden Lernmöglichkeiten: Lesen, Schreiben, Hörverständnis, Sprechen, Wortschatz, Grammatik, Dialoge und Prüfungen. Wir teilen die Bedienoberfläche also nicht künstlich nach einzelnen Sprachfertigkeiten auf.

Für Koreanisch und Japanisch entwickeln wir zusätzlich passende Schriftübungen für Hangul, Hiragana, Katakana und Kanji. Mit dem S Pen kannst du Schriftzeichen nachzeichnen, selbst schreiben und aus dem Gedächtnis wiedergeben. Die Strichfolge und Schriftzeichenerkennung können wir in weiteren Ausbaustufen prüfen.

Dazu kommen Sprachausgabe, einstellbare Wiedergabegeschwindigkeit, Hörtexte, Diktate, Sprachaufnahmen und die gewünschte **Aussprachekontrolle**. Für die detaillierte Bewertung von Lauten und Betonungen brauchen wir mehr als einfache Spracherkennung; deshalb entwickeln wir diese Funktion als eigenen technischen Teil innerhalb des Lernsystems.

Abgeschlossen, wenn: Du ein Sprachthema durch Lesen, Schreiben, Hören und Sprechen üben und deine Ergebnisse im selben Lernfortschritt nachverfolgen kannst.

Meilenstein 4 – Die Lernplattform funktioniert

Der Lernbereich vereint Lernbibliothek, Quizlet-ähnliche Methoden, Turm-Prüfmechanik, Runenschmiede, S-Pen-Eingabe und Sprachlernen. Ducky AI ist dafür noch keine Voraussetzung.

ABSCHNITT 05 · ENTWICKLUNGSPHASE

Ausbildung, Berufsschule, Job und Praktikum

Ziel: Der Planer begleitet dich auch nach einer Ausbildungs- oder Stellenzusage, ohne die bisherigen Bewerbungsinformationen zu verlieren.

19 Der freischaltbare Ausbildungsbereich

Dieser Bereich bleibt zunächst gesperrt. Sobald du einen Ausbildungsplatz erhalten hast, kannst du die Zusage bestätigen und den Ausbildungsbereich aktivieren. Eine vorläufige Zusage oder bloße Bewerbung darf ihn nicht unbeabsichtigt freischalten.

Wir entwickeln eine Verwaltung für den Ausbildungsvertrag, Betrieb, Ausbildungsberuf, Ausbildungsbeginn, Probezeit, Vergütung, Urlaub und weitere Vertragsbedingungen.

Die Berufsschule wird ebenfalls eingebunden: Stundenpläne, Blockunterricht, Fächer, Hausaufgaben, Prüfungen, Noten, Zeugnisse und Dokumente. Unterrichtstermine erscheinen im Kalender; Lernmaterialien nutzen unseren bereits vorhandenen Lernbereich.

Dazu kommt das digitale Berichtsheft mit täglichen oder wöchentlichen Tätigkeits- und Lernnachweisen. Bereits im Lernbereich vorhandene Nachweise können dafür verknüpft werden.

Die Ausbildungsvergütung und späteren Abrechnungen werden mit den Finanzen verbunden. Beendete Ausbildungsverhältnisse bleiben im Archiv erhalten.

Abgeschlossen, wenn: Eine Ausbildung von der bestätigten Zusage bis zu Berufsschule, Berichtsheft, Prüfungen und Vergütung vollständig organisiert werden kann.

20 Der Job- und Praktikumsbereich

Hier nutzen wir viele der bereits entwickelten Grundlagen erneut.

Ein Beschäftigungsverhältnis enthält Arbeitgeber, Art der Tätigkeit, Arbeitsvertrag, Zeitraum, Arbeitsort, Stundenumfang, Probezeit und Vertragsbedingungen.

Dazu entwickeln wir Dienstpläne, Schichtkalender, Zeiterfassung, Pausen, Fehlzeiten, Urlaub, Überstunden und ein Archiv für Lohnabrechnungen. Tatsächliche Zahlungen werden den Finanzen zugeordnet.

Für Praktika und lernintensive Tätigkeiten kommt das **Lern- und Tätigkeitsjournal** hinzu: tägliche Aufgaben, erworbene Kenntnisse, betreute Projekte, Feedback und Beurteilungen. Lerninhalte können wieder mit unserem gemeinsamen Lernbereich verbunden werden.

Mehrere Beschäftigungen sollen nebeneinander existieren können, etwa eine Ausbildung und ein Nebenjob. Jede hat eigene Vertrags-, Arbeitszeit- und Abrechnungsdaten.

Abgeschlossen, wenn: Du Job und Praktikum unabhängig von einer Ausbildung verwalten kannst und die relevanten Daten mit Kalender, Lernen und Finanzen verbunden sind.

ABSCHNITT 06 · ENTWICKLUNGSPHASE

Die Anwendung wird intelligent und zusammenhängend

Ziel: Die vielen Funktionen arbeiten miteinander, ohne dass du Informationen ständig mehrfach eingeben musst.

21 Dashboard, Verknüpfungen und Automatisierungen

Jetzt entwickeln wir eine zentrale Startseite, die aus deinen vorhandenen Daten die wichtigsten Informationen zusammenführt.

Sie zeigt beispielsweise den nächsten Termin samt geplanter Abfahrt, dringende Aufgaben, Bewerbungsfristen, Lernziele, bevorstehende Zahlungen und offene organisatorische Schritte.

Wir bauen Verknüpfungen aus: Bewerbungsgespräche zum Kalender, Reiseausgaben zu Finanzpositionen, Berufsschulinhalte zum Lernbereich, Arbeitszeiten zu Abrechnungen und Lerntagebucheinträge zu praktischen Nachweisen.

Außerdem ergänzen wir Benachrichtigungen, Wiedervorlagen, sinnvolle automatische Vorschläge und später einstellbare Regeln. Mit entsprechender Benutzerverwaltung können wir gemeinsame Kalender, geteilte Aufgaben und schließlich auch einen Mehrbenutzer-Chat entwickeln.

Die Oberfläche bleibt personalisierbar: Du bestimmst, welche Kategorien, Karten und Informationen du sehen möchtest.

Abgeschlossen, wenn: Du nicht mehr zwischen vielen unverbundenen Einzellösungen wechseln musst, sondern dein gesamter Alltag aus einem gemeinsamen Datenbestand organisiert wird.

22 Sichere Schnittstellen und Vorbereitung für Ducky AI

Wir bereiten die Anwendung darauf vor, mit KI-Systemen zu kommunizieren.

Dabei unterscheiden wir zwischen externen KI-Diensten, eventuell vorhandenen lokalen Modellen und einer späteren eigenen KI. Keine einzelne Plattform ist zwingende Voraussetzung.

Für eine KI-Anbindung definieren wir klar geregelte Schnittstellen: Termine lesen, Aufgaben vorschlagen, Lernmaterial bereitstellen, Aussprache bewerten und – nur mit passenden Berechtigungen – Änderungen vornehmen.

Ducky AI selbst bleibt dein eigenständiges, langfristiges Projekt. Ihre späteren Ziele gehen deutlich weiter: natürliche mehrsprachige Gespräche, Sprachverstehen, logisches Schlussfolgern, Gedächtnis und eigenständige Bearbeitung erlaubter Aufgaben.

Wir können diese Schnittstelle vorbereiten und irgendwann mit einer ersten KI verbinden, **ohne dass die gesamte Ducky AI bereits fertig entwickelt sein muss**.

Abgeschlossen, wenn: Unsere Anwendung eine dokumentierte, abgesicherte Schnittstelle besitzt, über die autorisierte KI-Funktionen angebunden und später ausgetauscht werden können.

ABSCHNITT 07 · ENTWICKLUNGSPHASE

Jetzt lohnen sich die Widgets

Ziel: Nicht bloß dekorative Kacheln, sondern schöne, praktische Erweiterungen unserer bereits funktionierenden Anwendung.

Zu diesem Zeitpunkt haben wir Kalender, Aufgaben, Finanzen, Bewerbungen und den Lernbereich mit echten Daten. Wir wissen also, **welche Informationen sich für kleine Displays wirklich eignen**.

23 Das Galaxy Z Flip8-Außendisplay

Zuerst entwickeln wir die Kacheln für das kleine Außendisplay des Flip8.

Kachel	Funktion
Mein Tag	Termine und Abfahrt
Aufgaben	Wichtig und dringend
Finanzen	Verfügbares Budget
Lernen	Nächstes Lernziel
Bewerbungen	Fristen und Rückmeldungen
Schnellaktionen	Schnell etwas erledigen

Wir lernen dafür zusätzlich die Android-Entwicklung mit Kotlin. Die Widgets werden **nicht bloß Webseiten im Kleinformat**, sondern native Android-Erweiterungen, die mit unserem Backend kommunizieren.

Samsung dokumentiert die Entwicklung eigener Widgets für das FlexWindow anhand des Flip5. Die tatsächlichen Möglichkeiten und Einschränkungen müssen wir beim Flip8 auf dem Gerät testen.

Dafür benötigen wir später eine Android-taugliche Entwicklungs- und Build-Umgebung. Dein Tab S8 bleibt dein Arbeitsplatz, allerdings ist Android Studio nicht einfach eine gewöhnliche Android-Tablet-Anwendung.

Abgeschlossen, wenn: Du echte Planerdaten auf dem Außendisplay sehen und geeignete Schnellaktionen zuverlässig ausführen kannst.

24 Die Galaxy Watch 8

Erst danach passen wir unsere wichtigsten Funktionen an die Uhr an.

Auf der Galaxy Watch entwickeln wir **Wear-OS-Kacheln (Tiles)** für kurze Informationen und schnelle Aktionen. Mögliche Kacheln zeigen den nächsten Termin, eine anstehende Abfahrt, dringende Aufgaben, das nächste Lernziel oder eine kurze Finanzübersicht.

Zusätzlich könnten wir Informationen auf dem Zifferblatt anzeigen, etwa die Zahl offener Aufgaben oder die Uhrzeit des nächsten Termins. Diese Anzeigen heißen *Complications*.

Die Oberfläche wird nicht einfach vom Flip8 übernommen: Auf der Uhr müssen Informationen noch knapper sein, ausreichend große Bedienelemente haben und auf dem runden Bildschirm gut lesbar bleiben. Auch die Aktualisierung und Zwischenspeicherung der Daten müssen wir beachten.

Abgeschlossen, wenn: Die Watch eigenständige, sinnvolle Kacheln mit Daten aus unserem Planer besitzt und die wichtigsten Aktionen schnell erreichbar sind.

25 Qualitätssicherung, Feinschliff und offizielle Fertigstellung

Zum Schluss nehmen wir uns die gesamte Webanwendung systematisch vor.

Wir überprüfen alle Funktionen, die Verbindungen zwischen den Bereichen, die Datenbank, Benutzerrechte, Dokumentenzugriffe, Fehlermeldungen, verschiedene Bildschirmgrößen und den sicheren Umgang mit persönlichen Informationen.

Außerdem testen wir den Alltagseinsatz: Was passiert ohne Internetverbindung? Was geschieht bei abgebrochenen Uploads? Werden wiederkehrende Termine korrekt verschoben? Funktionieren geplante Vertragsänderungen? Stimmen Finanzberechnungen auf den Cent? Werden Daten auf Tablet und Smartphone konsistent angezeigt?

Dazu kommen eine verständliche Bedienoberfläche, Tastatur- und S-Pen-Tests, Optimierung der Ladezeiten, Dokumentation und gegebenenfalls die Installation als PWA auf dem Startbildschirm. Eine PWA ist weiterhin eine Webanwendung und ersetzt unsere nativen Widgets nicht.

Zum Abschluss erstellen wir eine stabile Veröffentlichung auf dem Heimserver. Auch danach bleiben Wartung und neue Funktionen möglich.

Abgeschlossen, wenn: Die Anwendung nicht nur alle vorgesehenen Bereiche besitzt, sondern im tatsächlichen Alltag zuverlässig verwendet werden kann.

Meilenstein 5 – Die vollständige Anwendung

Der selbst entwickelte Alltagsplaner läuft auf eurem Server, verbindet die vereinbarten Lebensbereiche und besitzt Erweiterungen für Flip8 und Galaxy Watch 8. Ducky AI kann über die vorbereitete Schnittstelle weiterentwickelt und später integriert werden.

Wann welche Technologien wichtig werden

Zeitpunkt	Technologie	Wofür sie gebraucht werden
Anfang	HTML	Struktur der Webseite
Anfang	CSS	Einheitliches Design und responsive Oberflächen
Anfang	JavaScript	Interaktionen und Programmlogik
Früh	Git und GitHub	Versionsverwaltung und Kontrolle
Nach dem Mini-Projekt	Browser-Speicherung	Grundlagen dauerhafter Daten
Danach	Backend und Datenbank	Speicherung auf dem Heimserver, mehrere Geräte
Beim Ausbau	APIs, gegebenenfalls TypeScript/React	Saubere und erweiterbare Anwendung
Lernbereich	Canvas, Pointer Events	Echte S-Pen-Eingaben
Sprachen	Audioverarbeitung, Spracherkennung und Sprachausgabe	Hören, Sprechen, Aussprachekontrolle
Widgets	Kotlin, Android und Wear OS	Flip8-Außendisplay und Watch8
Langfristig	KI-Modelle und sichere KI-Schnittstellen	Externe KI-Dienste und Ducky AI

Nicht alle Programmiersprachen und Werkzeuge müssen vorher gelernt werden. Du lernst sie, sobald du sie für eine konkrete Funktion brauchen.

Leitlinien der weiteren Umsetzung

Bestehende Funktionen bleiben erhalten.

Die Entwicklung erfolgt in kleinen, nachvollziehbaren Schritten. Änderungen werden geprüft, bevor sie in die stabile Version übernommen werden.

Funktionen werden verbunden, nicht gedoppelt.

Der Lernbereich bleibt ein System. Ausbildung und Job verwenden dieselben vorhandenen Kalender-, Finanz- und Lernfunktionen, statt eigene Doppelversionen zu erzeugen.

Deine Daten bleiben unter deiner Kontrolle.

Der Heimserver verwaltet den dauerhaften Datenbestand und die Dokumente. Die Weboberfläche und spätere Widgets greifen nur mit passenden Berechtigungen darauf zu.

Du lernst das Entwickeln tatsächlich selbst.

Die Grundlagen werden systematisch erarbeitet. Programmierung, Planung, Fehlersuche und Tests bleiben nachvollziehbare eigene Lern- und Entwicklungsleistungen.

Der nächste konkrete Schritt

Schritt 1 – Die Entwicklungswerkstatt einrichten

Beginne mit GitHub, dem Projektverzeichnis und dem Editor auf deinem Tab S8. Danach folgt deine erste HTML-Seite.

Praktischer Einstieg: Ein Repository anlegen, den Codeeditor öffnen, Projektdateien prüfen, die erste HTML-Seite erstellen und Änderungen mit Git sichern.

Das ist die Arbeitsreihenfolge für das gesamte Projekt. Größere Funktionsziele lassen sich später noch detaillierter ausarbeiten, ohne die Gesamtstruktur jedes Mal neu erfinden zu müssen.